LECTURE 19

Constraint-based verification meets synthesis

19.1 Synthesis of invariants

Starting in the early 2000, researchers began to explore the idea of automatically discovering invariants by defining a parametric template of the likely invariant and then solving for the parameters in that template in order to make the verification condition valid. Early work in this direction focused on linear expressions, since methods for solving linear inequalities have been available for a while. Unfortunately, most programs require invariants that go beyond simple linear expressions. In 2007, Beyer, Henzinger, Majumdar and Rybalchenko [1] demonstrated how a more aggressive synthesis procedure could be used to generate conjunctive invariants, and in 2008, Gulwani, Srivastava and Venkatesan demonstrated an even more general synthesis technique to produce invariants with arbitrary boolean structure [2].

In general, the high-level idea is to generate a verification condition that includes an unknown invariant, and then search for a predicate that makes the verification condition valid.

For example, consider the following program:

```
assume y=y_0 && k=k_0 && t=y_0-k_0;
while(t>0){
           y = y-1;
           t = t-1;
}
assert y ≤ k_0;
```

So we want to prove that the assertion will be valid for any input that satisfies the assumption. From that program, we can generate a verification condition that will have the following form:

$$\forall y, y_0, k, k_0, t.(y = y_0 \wedge k = k_0 \wedge t = y_0 - k_0) \Rightarrow \begin{array}{l} inv(y, y_0, k, k_0, t) \wedge \\ \forall y, t. inv(y, y_0, k, k_0, t) \Rightarrow \left(\begin{array}{l} t > 0 \Rightarrow VC(\text{y = y-1; t = t-1}, inv(y, y_0, k, k_0, t)) \wedge \\ \neg(t > 0) \Rightarrow y \leq k_0 \end{array} \right) \end{array}$$

We do not actually know what the invariant is, but we hypothesize that it is a function of all the variables in scope, so we represent it as an unknown function inv. Now, $VC(\text{y = y-1; t = t-1}, inv(y, y_0, k, k_0, t)) = inv(y-1, y_0, k, k_0, t-1)$, and

the universal quantifier can be pulled out by renaming variables, which leaves us with the following verification condition.

$$\forall y, y_0, k, k_0, t, y', t'.(y = y_0 \wedge k = k_0 \wedge t = y_0 - k_0) \Rightarrow \begin{array}{l} inv(y, y_0, k, k_0, t) \wedge \\ inv(y', y_0, k, k_0, t') \Rightarrow \left(\begin{array}{l} t' > 0 \Rightarrow inv(y' - 1, y_0, k, k_0, t' - 1) \wedge \\ \neg(t' > 0) \Rightarrow y' \leq k_0 \end{array} \right) \end{array}$$

This is now a synthesis problem that can be solved using many of the synthesis techniques we have discussed so far. For example, in Sketch, you can encode the problem by making all the existentially quantified variables inputs to a test harness.

```
harness void main(int y, int y_0, int k, int k_0, int t,
                  int yp, int tp){
   if(y==y_0 && k==k_0 && t==y_0-k_0){
      assert inv(y, y_0, k, k_0, t);
   if(inv(yp, y_0, k, k_0, tp) && tp>0){
      assert inv(yp-1, y_0, k, k_0, tp-1);
   }
   if(inv(yp, y_0, k, k_0, tp) && tp≤0){
      assert yp ≤ k_0;
   }
} }
```

In Sketch, you can define the function inv to invoke a generator that will be replaced with an actual expression. For example, we can define inv as shown below.

```
bit inv(int y, int y_0, int k, int k_0, int t){
   return exprBool({y, y_0, k, k_0, t}, {PLUS});
}
```

The generator `exprBool` is defined in the standard library in a file named `generators.skh`. It will generate a predicate that uses the expressions passed in the first argument, and the operators passed as the second one. In order for the sketch to work, we need to include the following two lines in the header:

```
include ”generators.skh”;
pragma options ”--bnd-inline-amnt 2”;
```

The first line tells the synthesizer to include the `generators.skh` library, while the second line limits the search to expressions of depth two. We could give the synthesizer a bigger bound, but then we may get expressions that are bigger and more complicated than necessary. The result of running this sketch is shown below.

```
void inv (int y, int y_0, int k, int k_0, int t, ref bit _out)/*invariant.
    sk:15*/
{
  _out = y == (k_0 + t);
  return;
```

```
}
```

An important thing to note is that by default, sketch only checks this invariant against non-negative values of the input variables, and only within a given range. By default, Sketch considers only 5-bit inputs, although that can be increased using the flag `-bnd-inbits`. For this sketch, running with the default 5-bit inputs, it takes about 0.9 seconds to synthesize and verify. Increasing the number of input bits increases the verification time significantly; for example, for 8-bit inputs (which implies a total of 56 bits of input to the harness that has 7 separate integer inputs) the verification time grows to 27 seconds.

Another class of systems that can be used to solve problems like the one above are the SyGuS solvers. SyGuS is an effort by a number of researchers in the program synthesis community to develop a standard interface for synthesis problems.

19.2 Program Lifting

The ability to synthesize invariants in this way has implications beyond just verification. For example, in a paper back in 2013 led by Alvin Cheung [3], we explored the use of these kinds of techniques to prove that a fragment of Java code is equivalent to an SQL query. At a high-level, the approach was to take a block of Java code and generate a verification condition with unknown invariants *and* an unknown postcondition. The synthesizer was then asked to generate a post-condition that was of the form `output=SQL-Query`, and the loop invariants that would enable us to prove that post-condition. The strong requirement on the form of the postcondition prevented the synthesizer from generating trivial postconditions that could then be proven with trivial loop invariants.

The goal of proving this equivalence, between the loop and an SQL query, was to then replace the Java code with an equivalent query that could be executed directly in the database engine. This usually had the effect of reducing the amount of data that had to be transfered from the database to the web server, and in some cases it could even improve the asymptotic complexity of executing the query.

19.3 From verification conditions to synthesis conditions

Once we have the ability to synthesize unknown invariants, it is a relatively small step to synthesize the code itself. This was first explored by Srivastava, Gulwani and Foster in a paper that appeared in POPL 2010 [4]. They termed their approach "Proof Theoretic Synthesis" and they implemented it in a system called PTS.

The input to the system is a *Program Scaffold*, which includes a Flowgraph template that describes the nested loop structure of the desired program, a functional specification in the form of a pre and post-condition, as well as additional structural constraints such as limits on the number of temprary variables, the size of the different blocks of straight-line code, etc. The output of the system is a synthesized code fragment, together with the loop invariants and ranking function necessary to prove partial correctness and termination of the code.

The first important observation made by this work was that partial correctness is insufficient when the goal is to synthesize the bodies of loops, because it is extremely easy for the synthesizer to craft loop bodies and conditions that lead to infinite looping. This means that unless we force the synthesizer to also generate loop variants to prove termination of the synthesized code, most of what the synthesizer generates will fail to terminate.

Proof-Theoretic Synthesis

Input: Program Scaffold
- Flowgraph template (a nested loop structure)
- Function spec (pre-/post-conditions)
- Other constraints (temp variables, size limit, etc.)

Output:
- Synthesized code fragments
- Associated loop invariants and ranking functions

Synthesis Condition:
- Generalization of verification conditions
- A valid program exists iff. the SC has a satisfying solution

Example: Square Root

```
IntSqrt(int x) {
  v:=1;i:=1;
  while^{τ,φ}(v ≤ x)
    | v:=v+2i+1;i++;
  return i−1;
}
```

Invariant τ:
$v{=}i^2 \wedge x \geq (i{-}1)^2 \wedge i \geq 1$

Ranking function φ:
$x - (i{-}1)^2$

Program Scaffold

Pre: $x \geq 1$

$$
\begin{array}{l}
\texttt{choose}\,\{[]g_1 \rightarrow s_1\}\,; \\
\texttt{while}^{\tau,\varphi}\,(g_0)\,\{ \\
\quad \texttt{choose}\,\{[]g_2 \rightarrow s_2\}\,; \\
\}; \\
\texttt{choose}\,\{[]g_3 \rightarrow s_3\}
\end{array}
$$

Post: $(i-1)^2 \leq x < i^2$

(Assume only one branch in each fragment)

Verification Condition

$$
\begin{array}{l}
\texttt{choose}\,\{[]g_1 \rightarrow s_1\}\,; \\
\texttt{while}^{\tau,\varphi}\,(g_0)\,\{ \\
\quad \texttt{choose}\,\{[]g_2 \rightarrow s_2\}\,; \\
\}; \\
\texttt{choose}\,\{[]g_3 \rightarrow s_3\}
\end{array}
$$

$$
\begin{array}{rcll}
x \geq 1 \wedge g_1 \wedge s_1 & \Rightarrow & \tau' & \wedge \\
\tau \wedge g_0 \wedge g_2 \wedge s_2 & \Rightarrow & \tau' & \wedge \\
\tau \wedge \neg g_0 \wedge g_3 \wedge s_3 & \Rightarrow & (i'-1)^2 \leq x' \wedge x' < i'^2 &
\end{array}
$$

Well-formedness Condition

$$
\begin{array}{l}
\texttt{choose}\,\{[]g_1 \rightarrow s_1\}\,; \\
\texttt{while}^{\tau,\varphi}\,(g_0)\,\{ \\
\quad \texttt{choose}\,\{[]g_2 \rightarrow s_2\}\,; \\
\}; \\
\texttt{choose}\,\{[]g_3 \rightarrow s_3\}
\end{array}
$$

$$g_1, g_2, g_3 = \texttt{true}\ \textit{and}\ \texttt{valid}(s_1) \wedge \texttt{valid}(s_2) \wedge \texttt{valid}(s_3)$$

Ranking Function

$$
\begin{array}{l}
\texttt{choose}\,\{[]g_1 \rightarrow s_1\}\,; \\
\texttt{while}^{\tau,\varphi}\,(g_0)\,\{ \\
\quad \texttt{choose}\,\{[]g_2 \rightarrow s_2\}\,; \\
\}; \\
\texttt{choose}\,\{[]g_3 \rightarrow s_3\}
\end{array}
$$

$$r_l = \varphi_l \wedge (\tau \Rightarrow r_l \geq 0) \wedge (\tau \wedge g_0 \wedge g_2 \wedge s_2 \Rightarrow r_l' > \varphi_l')$$

Synthesis Conditions

$$sc = \texttt{SafetyCond}(exp, \mathcal{F}) \wedge \texttt{WellFormCond}(exp) \wedge \texttt{RankCond}(exp)$$

$$
\begin{array}{rcll}
x \geq 1 \wedge s_1 & \Rightarrow & \tau' & \wedge \\
\tau \wedge g_0 \wedge s_2 & \Rightarrow & \tau' & \wedge \\
\tau \wedge \neg g_0 \wedge s_3 & \Rightarrow & (i'-1)^2 \leq x' \wedge x' < i'^2 & \wedge \\
\texttt{valid}(s_1) \wedge \texttt{valid}(s_2) \wedge \texttt{valid}(s_3) & & & \wedge \\
r_l = \varphi_l \wedge (\tau \Rightarrow r_l \geq 0) \wedge (\tau \wedge g_0 \wedge s_2 \Rightarrow r_l' > \varphi_l') & & &
\end{array}
$$

Theorem (soundness and completeness)
The synthesis conditions are satisfiable iff. there exists a program that is valid w.r.t. the scaffold.

A second important design decision was to chose a representation that minimizes possible symmetries in the space of programs. One way they do that in this work is through their representation of loop-free blocks of code. Rather than allowing code fragments from a grammar involving assignments and conditionals, they use guarded parallel assignments of the form

$$\{g \rightarrow s\}$$

Where s corresponds to a set of assignments that all occur in parallel. The semantics of these guarded assignments first evaluate the guard, and if the guard is true then all the right-hand-sides are evaluated together, and then the values are all assigned to the left-hand-side variables in tandem. This means that the order of those assignments does not matter, unlike standard assignments in imperative programming languages. It also means that there is no nesting of conditionals.

There is another important implication of the use of these conditional parallel assignments in addition to reducing symmetries. The general technique assumes that we can generate verification conditions with some missing fragments, and it is up to the synthesizer to discover these fragments. However, it

is hard to generate verification conditions if we do not even know the structure of the code. The scaffold already gives us the loop structure of the program, so it allows us to define the verification condition in terms of the invariants and the verification condition of the bodies of the loops. However, if these boides were allowed to have arbitrary branching structure, it would be impossible to construct a verification condition when this structure is unknown. By canonicalizing everything to the conditional parallel assignments, it is now possible to define the verification conditions for these assignments when the conditions and the right-hand-sides are unknown.

References

[1] Dirk Beyer, Thomas A. Henzinger, Rupak Majumdar, and Andrey Rybalchenko. Invariant synthesis for combined theories. In Byron Cook and Andreas Podelski, editors, *Verification, Model Checking, and Abstract Interpretation*, pages 378–394, Berlin, Heidelberg, 2007. Springer Berlin Heidelberg. ISBN 978-3-540-69738-1. URL http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.65.4364&rep=rep1&type=pdf.

[2] Sumit Gulwani, Saurabh Srivastava, and Ramarathnam Venkatesan. Program analysis as constraint solving. In *Proceedings of the ACM SIGPLAN 2008 Conference on Programming Language Design and Implementation, Tucson, AZ, USA, June 7-13, 2008*, pages 281–292, 2008. doi: 10.1145/1375581.1375616. URL http://doi.acm.org/10.1145/1375581.1375616.

[3] Alvin Cheung, Armando Solar-Lezama, and Samuel Madden. Optimizing database-backed applications with query synthesis. In *ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI '13, Seattle, WA, USA, June 16-19, 2013*, pages 3–14, 2013. doi: 10.1145/2462156.2462180. URL http://doi.acm.org/10.1145/2462156.2462180.

[4] Saurabh Srivastava, Sumit Gulwani, and Jeffrey S. Foster. From program verification to program synthesis. In *Proceedings of the 37th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2010, Madrid, Spain, January 17-23, 2010*, pages 313–326, 2010. doi: 10.1145/1706299.1706337. URL http://doi.acm.org/10.1145/1706299.1706337.